Rapport

table des matière

1. introduction
    - 1.1 objectifs produit
    - 1.2 pédagogique
    - 1.3 Description du domaine
2. analyse fonctionnelle
3. planification initiale
    - 3.1Macro-Planning
4. Rapport de tests
5. Utilisation de L'IA
6. Bilan de déroulement
7. Bilan produit
8. Conclusion

introduction

objectifs produit

Le projet Plot Those Lines (PTL) vise à concevoir une application dédiée à la visualisation et à l'analyse de la météo spatiale. L'application permet d'importer, de stocker localement et d'afficher simultanément jusqu'à 5 séries de données complexes sur un axe temporel commun. Grâce à une interface graphique flexible et un mode de fonctionnement 100 % hors-ligne.

pédagogique

Ce projet permet de pratiquer et de valider les notions théoriques et techniques vues en cours

Description du domaine

Le domaine d'application retenu pour ce projet est la météo spatiale, et plus particulièrement la surveillance des variations du champ magnétique interplanétaire (IMF) mesuré au point de Lagrange L1 entre le Soleil et la Terre (via le satellite DSCOVR de la NOAA).

La météo spatiale étudie l'impact de l'activité solaire sur l'environnement spatial terrestre. Lors d'éruptions solaires ou d'éjections de masse coronale, un vent solaire chargé en particules magnétisées frappe la la Terre. Ces événements peuvent perturber les satellites de télécommunication, dégrader les signaux GPS, créer des surtensions sur les réseaux électriques haute tension et générer des aurores polaires.

Cohérence d'affichage

Puisque les séries B_x_gse, B_y_gse, B_x_gsm, B_y_gsm et B_t partagent la même unité physique (le nanotesla, nT), elles peuvent être superposées directement sur un axe Y unique sans déformer l'affichage.

L’analyse fonctionnelle

#1 — Stockage local des séries temporelles

En tant qu' utilisateur, je veux que l'ensemble des séries importées soit sauvegardé localement sur mon poste, afin de pouvoir relancer le logiciel et travailler sur mes données en mode hors-ligne.

- ☐ GIVEN des données importées avec succès, WHEN

l'application est fermée puis rouverte, THEN les 5 séries temporelles sont automatiquement rechargées.
- ☐ GIVEN une utilisation sans connexion réseau, WHEN l'application démarre, THEN toutes les séries enregistrées localement restent accessibles.

#3 — Affichage graphique simultané des 5 séries (ScottPlot)

En tant qu' utilisateur, je veux afficher une représentation graphique de 5 séries du champ magnétique sur un axe temporel commun, afin de comparer visuellement leurs variations en nanoteslas (nT) sur la même période.

[image]

- ☐ GIVEN 5 séries enregistrées, WHEN l'utilisateur ouvre la vue principale, THEN le graphique affiche les 5 courbes avec des couleurs distinctes.
- ☐ GIVEN le graphique affiché, WHEN l'utilisateur observe l'axe horizontal (X), THEN l'axe représente le temps (timestamps synchronisés).
- ☐ GIVEN le graphique affiché, WHEN l'utilisateur observe l'axe vertical (Y), THEN l'échelle est exprimée en nanoteslas (nT) pour toutes les séries.

#4 — Sélection et filtrage dynamique des séries

En tant qu' utilisateur, je veux pouvoir cocher ou décocher chaque série via des cases à cocher (Checkboxes), afin de masquer les courbes inutiles et d'analyser en détail seulement 2 ou 3 séries spécifiques.

[image]

- ☐ GIVEN un graphique avec 5 séries actives, WHEN l'utilisateur décoche Bz_gsm, THEN la courbe correspondante disparaît

immédiatement du graphique.
- ☐ GIVEN une série masquée, WHEN l'utilisateur coche à nouveau la case, THEN la courbe réapparaît exactement à sa place sans recharger l'ensemble des données.

#5 — Ajout manuel d'une mesure à une série temporelle

[BONUS/si le temps le permet]

En tant qu' utilisateur, je veux pouvoir ajouter manuellement une nouvelle valeur datée à une série existante, afin de compléter mes données locales si une mesure manque.

[image]

- ☐ GIVEN une série sélectionnée, WHEN l'utilisateur saisit une date et une valeur valide puis valide, THEN le nouveau point est inséré dans le stockage local.
- ☐ GIVEN un nouveau point ajouté, WHEN le graphique se rafraîchit, THEN le point apparaît à la bonne position chronologique sur la courbe.
- ☐ GIVEN une saisie avec une valeur non numérique, WHEN l'utilisateur valide, THEN un message d'avertissement bloque la saisie.

---

vous pouvez vois les US avec les lien suivant :

- US-1 : https://github.com/Gianmarco-Ruberti/P_FUN-Gianmarco/issues/1#issue-5232498212
- US-2 : https://github.com/Gianmarco-Ruberti/P_FUN-Gianmarco/issues/2#issue-5232526328
- US-3 : https://github.com/Gianmarco-Ruberti/P_FUN-Gianmarco/issues/3#issue-5232601596
- US-4 : https://github.com/Gianmarco-Ruberti/P_FUN-Gianmarco/issues/4#issue-5232617330

La planification initiale

La réalisation du projet s'étale sur **24 périodes** réparties du début du trimestre au **30 octobre 2026** (date de livraison finale).

Macro-Planning du projet

Phase	Périodes estimées	Activités principales	Livrable associé
1. Analyse & Prise en main	**4 périodes**	Choix du domaine, rédaction des US, validation du Kanban	**Release v1 (04.09.2026)**
2. Modèle & Parsing JSON	**4 périodes**	Création des classes C# (`DataPoint`, `Series`), lecture de `solar_mag.json` avec LINQ et stockage local.	Commits branche `main`
3. Interface & ScottPlot	**9 périodes**	Apprentissage de ScottPlot, création de l'interface WPF, affichage des 5 courbes et gestion des Checkboxes.	Commits branche `main`
4. Fonctionnalités & Tests	**4 périodes**	Ajout d'une mesure (US-05), création des 2 méthodes d'extension C#, écriture de 3 tests unitaires et corrections.	Rapport de tests

5. Bilan & Documentation	**3 périodes**	Rédaction de la section sur l'IA, mise au propre du journal de travail et préparation de la release finale.	**Release Finale (30.10.2026)**

Rapport de tests

L'obectif de ces test est de validé le fonctionnement de la méthode `LoadFromFile` lors du chargement des donnéees depuis un fichier JSON les 3 test valide les points suivant :

1. les données valides issues du satéllite **SOLAR1** sont extraites et converties
2. les données d'autres source sont ignorées
3. le système donne les bonne exceptions en cas d'absence de données de **SOLAR1**

ID Test	nom du test	Scénario de Test	Résultat a
TU-01	`LoadFromFile_ValidDataTests`	Chargement d'un fichier JSON contenant des données valide pour `SOLAR1`	Le fichier es erreur, 1 élé retourné avec c `y_gse`, `bz_g` conform

TU-02	`LoadFromFile_FilteringTests`	Chargement d'un fichier JSON contenant une donnée valide pour `SOLAR1` et une donnée de `ACE`	la donnée de ignorée et la d `SOLAR1` est c
TU-03	`LoadFromFile_ExceptionTests`	Chargement d'un fichier JSON contenant uniquement une donnée de `ACE`	`InvalidDataE` est levée en l'absence de valide

TU-01 : Validation du Cas Nominal (LoadFromFile_ValidDataTests.cs)

- Description : S'assure que les données extrait du JSON ont les propriétés requises (`ByGse`, `BzGse`, `Bt`, `TimeTag`) lorsque la source est SOLAR1.
- Couverture : Méthode LoadFromFile et méthode auxiliaire GetDoubleProperty.

TU-02 : Validation du Filtrage (LoadFromFile_FilteringTests.cs)

- Description : Vérifie le respect des règles concernant la priorité et l'exclusion des satellites autre que **SOLAR1**.
- Couverture : Condition de filtrage source == "SOLAR1".

TU-03 : Validation de la Gestion des Erreurs (LoadFromFile_ExceptionTests.cs)

- Description : S'assure que l'application ne traite pas un fichier invalide ou sans les données demander.
- Couverture : Bloc de validation final hasValidData et levée d'exception InvalidDataException.

Utilisation de L'IA dans ce projet

Cette section décrit l'usage réel de l'IA sur le projet : l'outil unique utilisé, son mode d'utilisation, un exemple concret et ce que j'ai vérifié ou corrigé avant d'intégrer les modifications dans le projet.

Outils et modes d'utilisation

Pour la réalisation de ce projet et la rédaction de ce rapport, j'ai eu recours à un unique outil : Gemini, utilisé exclusivement via son interface de chat en ligne.

- Assistance technique : Conception de la logique de filtrage (ciblage strict de la source SOLAR1).
- Assistance rédactionnelle : Relecture, correction orthographique, reformulation et structuration globale du rapport technique.

La démarche n'était pas de laisser l'IA faire tout le travail de manière aveugle, mais de l'utiliser comme un assistant de codage et de relecture. Chaque bloc de code proposé a dû être analysé, intégré manuellement et testé pour s'assurer qu'il fonctionnait correctement dans la solution.

Exemple concret d'utilisation

Exemple 1 — Refactorisation et explication de la méthode de parsing GetDoubleProperty

Demande / Prompt :

```
private double? GetDoubleProperty(JsonElement element, st

{

    if (!element.TryGetProperty(propertyName, out JsonEle

        return null;


    if (propElem.ValueKind == JsonValueKind.Number && pro

        return valNum;


    if (propElem.ValueKind == JsonValueKind.String &&

        double.TryParse(propElem.GetString(), NumberStyle

        return valStr;


    return null;

}
```

Réponse de l'IA :

Voici l'explication de cette version condensée de la méthode GetDoubleProperty : Rôle général

Cette méthode extrait la valeur numérique d'une propriété spécifique dans un objet JSON (JsonElement) et la retourne sous forme de double? (un nombre décimal pouvant être nul). Elle gère aussi bien les nombres stockés directement en tant que types numériques JSON que ceux stockés sous forme de chaînes de caractères (ex: "12.34"). Explication pas à pas

1. Vérification de l'existence et des valeurs nulles

```
if (!element.TryGetProperty(propertyName, out JsonElement
    return null;
```

Cette ligne combine deux contrôles :

TryGetProperty : Tente de trouver la propriété nommée propertyName. Si elle n'existe pas dans l'objet JSON, la méthode s'arrête immédiatement et renvoie null.

ValueKind == JsonValueKind.Null : Si la propriété existe mais contient explicitement null (ex: "by_gse": null), elle renvoie également null.

2. Extraction si la valeur est un nombre

```
if (propElem.ValueKind == JsonValueKind.Number && propEle
    return valNum;
```

Si le JSON contient un nombre natif (ex: "bt": 5.4), propElem.ValueKind vaut Number.

TryGetDouble convertit directement cet élément JSON en type double C# et le renvoie.

3. Extraction si la valeur est une chaîne de texte

```
if (propElem.ValueKind == JsonValueKind.String &&
    double.TryParse(propElem.GetString(), NumberStyles.An
    return valStr;
```

Si la valeur est entourée de guillemets dans le JSON (ex: "bt": "5.4"), propElem.ValueKind vaut String.

double.TryParse tente de convertir le texte en double.

CultureInfo.InvariantCulture garantit que le point (.) est bien utilisé comme séparateur décimal, indépendamment des paramètres régionaux de l'ordinateur qui exécute le programme.

4. Valeur par défaut

```
return null;
```

Si la propriété existe mais n'est ni un nombre valide ni un texte convertible en nombre (par exemple un boooléen true ou un tableau []), la méthode retourne null. Ce qui change par rapport à la version précédente

C'est une refactorisation plus propre :

```
Les vérifications d'existence (TryGetProperty) et de vale
```

Ce qui a été compris ou corrigé :

J'ai compris l'utilité de fusionner les conditions de sécurité (TryGetProperty et Null) dès le début de la méthode pour éviter d'alourdir le code. J'ai également validé l'importance d'utiliser CultureInfo.InvariantCulture pour garantir que le point décimal est correctement lu peu importe la configuration linguistique de l'ordinateur qui exécute l'application.

Exemple 2 — coorection de l'orthographe sans en changer les mots

Demande / Prompt :

le planning initial n'a pas été réspecter a 100% car je ne j'avais surrésitimé cèraine tâches, sous esstimer d'autre et après avoir améloré mais connaissance technique j'ai deplacer le stockage local a la fin pour une meilleur fluidité mais en général le planning a été respécté

Réponse de l'IA :

Voici votre texte avec l'orthographe et la grammaire corrigées, sans avoir changé ni supprimé de mots :

Le planning initial n'a pas été respecté à 100% car je n'avais surestimé certaines tâches, sous-estimé d'autres et après avoir amélioré mes

connaissances techniques j'ai déplacé le stockage local à la fin pour une meilleure fluidité mais en général le planning a été respecté.

Ce qui a été compris ou corrigé :

J'ai utilisé l'IA strictement comme un outil de correction orthographique et grammaticale de base (avec la consigne implicite de ne pas réécrire mon style). J'ai vérifié la proposition pour m'assurer qu'elle corrigeait mes fautes de frappe (réspecter -> respecté, surrésitimé -> surestimé, cèraine -> certaines, etc.) tout en préservant exactement mes propres mots et ma tournure de phrase originale.

Bilan critique et prise de recul

Après discussion avec mon professeur, j'ai pris conscience d'un écueil important dans ma méthode de travail : l'utilisation que j'avais de l'IA pour comprendre ou corriger le code était parfois trop sommaire. En obtenant des solutions prêtes à l'emploi trop rapidement, cela ne m'a pas permis d'ancrer les connaissances sur le long terme, privilégiant une résolution immédiate à court terme. Cette prise de recul m'a permis de mettre en lumière les limites de ma méthode d'apprentissage face à ces assistants, et m'incite à approfondir par moi-même la logique interne du code pour mes futurs projets.

Bilan du déroulement de projet

Respect du planning

Bien que le planning initial n'ait pas été entièrement respecté, l'ensemble du projet est resté dans les temps. La réorganisation de certaines tâches s'explique par une réévaluation de leur durée et par l'acquisition de nouvelles connaissances techniques, qui m'ont incité à décaler le stockage local à la fin pour optimiser le travail.

Méthodologie

J'ai appliqué la méthodologie agile Scrum à ce projet. la phase de débriefing en début de session s'est révélée très bénéfique : elle m'a permit de faire le point sur l'avancement avant de me lancer dans le code.

Problèmes rencontré

Le principal obstacle rencontré résidait dans l'apparition de données "fantômes". Celles-ci provenaient d'un doublon lié à une double source de données dans le fichier JSON. Pour y remédier, j'ai mis en place un filtrage préalable afin de ne conserver qu'une seule source, amplement suffisante pour l'analyse.

Bilan produit

7.1 Prévu / réalisé

Fonctionnalité	Prévue	Réalisée	Commentaire
Stockage local des séries temporelles (US1)	Oui	Oui	L'objectif est d'enregistrer automatiquement les 5 séries temporelles importées sur le poste de l'utilisateur pour qu'elles restent accessibles et rechargeables au redémarrage, même sans connexion réseau.

Importation des données du champ magnétique (US2)	Oui	Oui	L'objectif est d'extraire de manière sécurisée et hors-ligne 5 séries temporelles spatiales de plus de 500 points chacune, tout en gérant proprement les cas de fichiers corrompus avec un message d'erreur.
Affichage graphique simultané des 5 séries (US3)	Oui	Oui	L'objectif est d'afficher sur un graphique unique les courbes distinguées par couleur, partageant un axe temporel commun et une échelle unifiée en nanoteslas
Sélection et filtrage dynamique des séries (US4)	Oui	Oui	L'objectif est de permettre à l'utilisateur de masquer ou réafficher instantanément chaque courbe sur le graphique sans recharger les données.

Fonctions ajoutées, non prévues initialement

Ces fonctions ne sont dans aucune user story. Elles sont nées des problèmes rencontrés pendant le développement :

Ajout	**Raison**
filtre des source de données	a cause d'une double source de données il y avais de données phantome

Améliorations possibles

Malgré l'atteinte des objectifs fixés et le bon fonctionnement de l'application, plusieurs pistes d'amélioration pourraient être apportées pour faire évoluer le projet :

- Architecture Data-Driven centralisée : Bien que demandée lors de l'évaluation à 80 %, cette approche n'a pas pu être intégrée dans la branche principale. En effet, je ne comprenais pas suffisamment bien le code existant pour réaliser le merge en toute confiance. Donc, comme convenu lors de notre discussion, je l'ai laissée dans une autre branche et j'ai rédigé ce paragraphe pour expliquer la raison de son absence.

- Connexion en direct aux flux de données de la NOAA : L'application repose actuellement sur une importation manuelle de fichiers JSON ou l'utilisation d'un stockage local. Une évolution majeure consisterait à intégrer un service réseau capable d'interroger automatiquement l'API en ligne de la NOAA pour récupérer les dernières mesures du satellite DSCOVR en temps réel.

Conclusion

Au cours de ce projet, j'ai pu concevoir une application dédiée à la visualisation graphique en partant de demandes précises en termes de fonctionnalités : le stockage local, l'importation, le filtrage et l'affichage simultané de séries temporelles, pour lesquelles j'ai choisi le thème du champ magnétique interplanétaire. Ce projet m'a offert un bon cadre pour mettre en pratique les compétences de LINQ vues pendant le module.

Sur le plan personnel, ce projet m'a permis de comprendre l'importance d'une bonne structure de code et d'une bonne gestion des imprévus, comme la gestion des doublons de sources. Les discussions sur le code pendant ce projet m'ont aussi permis de souligner les défauts de ma manière d'apprentissage, qui était trop limitée dans le temps lors de l'utilisation de l'IA.

En somme, PTL a été un projet que j'ai apprécié et qui m'a permis de mieux gérer mes projets, de mieux comprendre la programmation et d'améliorer ma méthode d'apprentissage avec l'assistance de l'IA